

МОСКОВСКИЙ ГОРОДСКОЙ ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ

Институт цифрового образования

Практическая работа 2.1

«Разработка и тестирование dbt-моделей для бизнес-логики»

Выполнил студент 1 курса, БД-251м группы

Трусов А.И.

Москва – 2015

Архитектура DWH



Рисунок 1 - Архитектура DWH

Ключевые фрагменты кода

Код модели stg_orders.sql

```
-- models/staging/stg_orders.sql
-- Эта модель читает данные из исходной таблицы stg.orders,
-- приводит их к нужным типам и исправляет ошибку с почтовым кодом.
-- Все последующие модели будут ссылаться на эту, а не на исходную таблицу.

SELECT
  -- Приводим все к нижнему регистру для консистентности в dbt
  "order_id",
  ("order_date")::date as order_date,
  ("ship_date")::date as ship_date,
  "ship_mode",
  "customer_id",
  "customer_name",
  "segment",
  "country",
  "city",
  "state",
  -- Исправляем проблему с Burlington прямо здесь, один раз и навсегда
  CASE
    WHEN "city" = 'Burlington' AND "postal_code" IS NULL THEN '05401'
    ELSE "postal_code"
  END as postal_code,
  "region",
  "product_id",
  "category",
  "subcategory" as sub_category, -- переименовываем для соответствия
  "product_name",
  "sales",
  "quantity",
  "discount",
  "profit"
FROM {{ source('stg', 'orders') }}
```

Код модели sales_fact.sql

```
-- Создает таблицу фактов, объединяя все измерения
SELECT
  -- Суррогатные ключи из измерений
  cd.cust_id,
  pd.prod_id,
  sd.ship_id,
  gd.geo_id,
  -- Ключи для календаря
  to_char(o.order_date, 'yyyymmdd')::int AS order_date_id,
  to_char(o.ship_date, 'yyyymmdd')::int AS ship_date_id,
  -- Бизнес-ключ и метрики
  o.order_id,
  o.sales,
  o.profit,
  o.quantity,
  o.discount
FROM {{ ref('stg_orders') }} AS o
LEFT JOIN {{ ref('customer_dim') }} AS cd ON o.customer_id = cd.customer_id
LEFT JOIN {{ ref('product_dim') }} AS pd ON o.product_id = pd.product_id
LEFT JOIN {{ ref('shipping_dim') }} AS sd ON o.ship_mode = sd.ship_mode
LEFT JOIN {{ ref('geo_dim') }} AS gd ON o.postal_code = gd.postal_code AND o.city =
gd.city AND o.state = gd.state
```

Код модели mart_office_supplies_drilldown.sql (вариант 29)

```
-- models/marts/mart_office_supplies_drilldown.sql
SELECT
  p.sub_category,
  SUM(f.profit) AS total_profit
FROM {{ ref('sales_fact') }} AS f
LEFT JOIN {{ ref('product_dim') }} AS p ON f.prod_id = p.prod_id
WHERE p.category = 'Office Supplies'
GROUP BY p.sub_category
ORDER BY total_profit DESC
```

Файл schema.yml

```
# Путь к файлу: models/marts/schema.yml
```

```
version: 2
```

```
models:
```

```
- name: shipping_dim
```

```
  columns:
```

```
    - name: ship_id
```

```
    tests:
```

```
      - unique
```

```
      - not_null
```

```
- name: customer_dim
```

```
  columns:
```

```
    - name: cust_id
```

```
    tests:
```

```
      - unique
```

```
      - not_null
```

```
- name: geo_dim
```

```
  columns:
```

```
    - name: geo_id
```

```
    tests:
```

```
      - unique
```

```
      - not_null
```

```
- name: product_dim
```

```
  columns:
```

```
    - name: prod_id
```

```
    tests:
```

```
      - unique
```

```
      - not_null
```

```
- name: sales_fact
```

```
  columns:
```

```
    - name: cust_id
```

```
    tests:
```

```
      - relationships:
```

```
        arguments:
```

```
          to: ref('customer_dim')
```

```
          field: cust_id
```

```
- name: mart_office_supplies_drilldown
```

```
  columns:
```

```
    - name: sub_category
```

```
    tests:
```

```
      - not_null
```

```
      - unique
```

```
    - name: total_profit
```

```
    tests:
```

```
      - not_null
```

Результаты

```
(dbt-env) dev@dev-vm:~/Downloads/pde_magistr/superstore_dwh$ dbt run
15:10:00 Running with dbt=1.10.11
15:10:00 Registered adapter: postgres=1.9.1
15:10:02 Found 8 models, 12 data tests, 1 source, 435 macros
15:10:02 Concurrency: 4 threads (target='dev')
15:10:02
15:10:02 1 of 8 START sql table model dw_test.calendar_dim ..... [RUN]
15:10:02 2 of 8 START sql view model stg.stg_orders ..... [RUN]
15:10:02 2 of 8 OK created sql view model stg.stg_orders ..... [CREATE VIEW in 0.43s]
15:10:02 3 of 8 START sql table model dw_test.customer_dim ..... [RUN]
15:10:02 4 of 8 START sql table model dw_test.geo_dim ..... [RUN]
15:10:02 5 of 8 START sql table model dw_test.product_dim ..... [RUN]
15:10:03 1 of 8 OK created sql table model dw_test.calendar_dim ..... [SELECT 7670 in 0.64s]
15:10:03 6 of 8 START sql table model dw_test.shipping_dim ..... [RUN]
15:10:03 3 of 8 OK created sql table model dw_test.customer_dim ..... [SELECT 1093 in 0.39s]
15:10:03 4 of 8 OK created sql table model dw_test.geo_dim ..... [SELECT 932 in 0.38s]
15:10:03 5 of 8 OK created sql table model dw_test.product_dim ..... [SELECT 4644 in 0.40s]
15:10:03 6 of 8 OK created sql table model dw_test.shipping_dim ..... [SELECT 4 in 0.24s]
15:10:03 7 of 8 START sql table model dw_test.sales_fact ..... [RUN]
15:10:03 7 of 8 OK created sql table model dw_test.sales_fact ..... [SELECT 25967 in 0.33s]
15:10:03 8 of 8 START sql table model dw_test.mart_office_supplies_drilldown ..... [RUN]
15:10:03 8 of 8 OK created sql table model dw_test.mart_office_supplies_drilldown ..... [SELECT 9 in 0.09s]
15:10:03
15:10:03 Finished running 7 table models, 1 view model in 0 hours 0 minutes and 1.60 seconds (1.60s).
15:10:04
15:10:04 Completed successfully
15:10:04
15:10:04 Done. PASS=8 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=8
```

Рисунок 2 - Скриншот успешного выполнения dbt run

```
(dbt-env) dev@dev-vm:~/Downloads/pde_magistr/superstore_dwh$ dbt test
15:11:31 Running with dbt=1.10.11
15:11:31 Registered adapter: postgres=1.9.1
15:11:33 Found 8 models, 12 data tests, 1 source, 435 macros
15:11:33 Concurrency: 4 threads (target='dev')
15:11:33
15:11:33 1 of 12 START test not_null_customer_dim_cust_id ..... [RUN]
15:11:33 2 of 12 START test not_null_geo_dim_geo_id ..... [RUN]
15:11:33 4 of 12 START test not_null_mart_office_supplies_drilldown_total_profit ..... [RUN]
15:11:33 3 of 12 START test not_null_mart_office_supplies_drilldown_sub_category ..... [RUN]
15:11:33 2 of 12 PASS not_null_geo_dim_geo_id ..... [PASS in 0.27s]
15:11:33 5 of 12 START test not_null_product_dim_prod_id ..... [RUN]
15:11:33 1 of 12 PASS not_null_customer_dim_cust_id ..... [PASS in 0.31s]
15:11:33 3 of 12 PASS not_null_mart_office_supplies_drilldown_sub_category ..... [PASS in 0.26s]
15:11:33 6 of 12 START test not_null_shipping_dim_ship_id ..... [RUN]
15:11:33 4 of 12 PASS not_null_mart_office_supplies_drilldown_total_profit ..... [PASS in 0.34s]
15:11:33 7 of 12 START test relationships_sales_fact_cust_id_cust_id_ref_customer_dim ..... [RUN]
15:11:33 8 of 12 START test unique_customer_dim_cust_id ..... [RUN]
15:11:34 5 of 12 PASS not_null_product_dim_prod_id ..... [PASS in 0.23s]
15:11:34 9 of 12 START test unique_geo_dim_geo_id ..... [RUN]
15:11:34 6 of 12 PASS not_null_shipping_dim_ship_id ..... [PASS in 0.27s]
15:11:34 7 of 12 PASS relationships_sales_fact_cust_id_cust_id_ref_customer_dim ..... [PASS in 0.27s]
15:11:34 10 of 12 START test unique_mart_office_supplies_drilldown_sub_category ..... [RUN]
15:11:34 11 of 12 START test unique_product_dim_prod_id ..... [RUN]
15:11:34 8 of 12 PASS unique_customer_dim_cust_id ..... [PASS in 0.25s]
15:11:34 12 of 12 START test unique_shipping_dim_ship_id ..... [RUN]
15:11:34 9 of 12 PASS unique_geo_dim_geo_id ..... [PASS in 0.18s]
15:11:34 10 of 12 PASS unique_mart_office_supplies_drilldown_sub_category ..... [PASS in 0.15s]
15:11:34 11 of 12 PASS unique_product_dim_prod_id ..... [PASS in 0.15s]
15:11:34 12 of 12 PASS unique_shipping_dim_ship_id ..... [PASS in 0.11s]
15:11:34
15:11:34 Finished running 12 data tests in 0 hours 0 minutes and 1.12 seconds (1.12s).
15:11:34
15:11:34 Completed successfully
15:11:34
15:11:34 Done. PASS=12 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=12
```

Рисунок 3 - Скриншот успешного выполнения dbt test

Выводы

В ходе выполнения практической работы по разработке DWH с использованием dbt я на собственном опыте оценил ключевые преимущества этого подхода по сравнению с традиционным написанием DDL/DML-скриптов вручную.

Основные преимущества, которые я выделил:

Автоматизация рутинных процессов — вместо того чтобы вручную прописывать порядок выполнения SQL-скриптов и следить за зависимостями, dbt сам определяет правильную последовательность сборки моделей. Это особенно ценно при работе с большим количеством таблиц-измерений и фактов.

Встроенное тестирование данных — возможность декларативно описать тесты прямо в YAML-файлах и запускать их одной командой `dbt test` кардинально меняет подход к качеству данных. Я смог легко добавить проверки на уникальность, полноту и целостность связей между таблицами.

Прозрачность и документирование — генерация графа зависимостей (lineage) в `dbt docs` наглядно показывает, как данные преобразуются от сырых таблиц до готовых витрин. Это неоценимо для понимания архитектуры хранилища новыми членами команды.

Модульность и переиспользование — использование ссылок через `{{ ref() }}` вместо прямых ссылок на таблицы делает код более переносимым и безопасным. Макросы позволяют избежать дублирования сложной бизнес-логики.

Контроль версий и CI/CD — вся структура DWH хранится в виде кода в Git, что позволяет отслеживать изменения, проводить код-ревью и автоматизировать развертывание.